

Infosys Springboard Virtual Internship 6.0

Project Title : Code Gen AI & Code Explainer

Presented By: Ch. Gayatri

Mentor: *Rojar*

Introduction

With the rapid advancement of **Artificial Intelligence (AI)**, developers increasingly rely on AI-assisted coding platforms to improve productivity and understanding.□

However, most tools today depend on **cloud-based APIs**, which can be slow, costly, or privacy-sensitive.

To overcome these limitations, the **CodeGen AI & Code Explainer** was developed as a **local, self-contained intelligent assistant**. It leverages:

- **Ollama** — a local LLM engine that runs models like *Llama2*, *Gemma*, and *Mistral* on personal computers, and
- **Streamlit** — an open-source Python framework for rapid web app development.

The integration enables:

- Real-time **streaming of AI responses**,
- **Code generation** tailored to user prompts, and
Instant code explanations for learning or debugging.

The application also supports **multi-session chats**, **searchable history**, and **model selection**, making it a complete interactive AI workspace for coders.

Outline

1. Project Overview
2. Problem Statement
3. Abstract
4. Objectives
5. Technologies Used
6. System Architecture
7. Integration of Ollama (with Code Snippets)
8. Streamlit Frontend Design
9. Data Management
10. Features
11. Challenges & Error Fixes (Real Issues Faced)
12. Deployment Steps
13. Key Learnings
14. Conclusion
15. References

1. Project Overview

Project Name: CodeGen AI & Code Explainer

Purpose:□

To design and develop an **AI-powered code assistant** that helps users generate and explain code through a **ChatGPT-style Streamlit interface** using **Ollama** (local LLMs).□

This ensures **offline operation**, **data privacy**, and **high performance**.

Provide natural-language → code generation across popular languages (Python, JavaScript/TypeScript, Java, Go, Rust, C#, SQL).

Explain existing code blocks with variable-level, function-level and architectural explanations.

Offer refactoring suggestions, unit test generation, and inline code comments.

Run fully offline via Ollama (or equivalent local model runtime) to ensure privacy and control.

Provide a Streamlit UI that is simple, conversational, and developer-friendly.

Enable plugin-like integrations for linters, formatters, and test runners locally.

2.Problem Statement

Developers often need quick, contextual help while writing or understanding code.□

Switching between tools for explanation, generation, and debugging reduces productivity.

Goal: Build a unified, interactive, *AI-powered code assistant* that runs locally and helps developers:

- Explain existing code intelligently.

3. Abstract

The **CodeGen AI & Code Explainer** integrates **Streamlit (Frontend)** and **Ollama (Backend LLM Engine)**.□

The app interacts with locally stored AI models like **Llama2**, **Gemma**, and **Mistral**, supporting both **code generation** and **explanation**.

Each chat session is stored in chats.json for persistence and future access.

4.Objectives

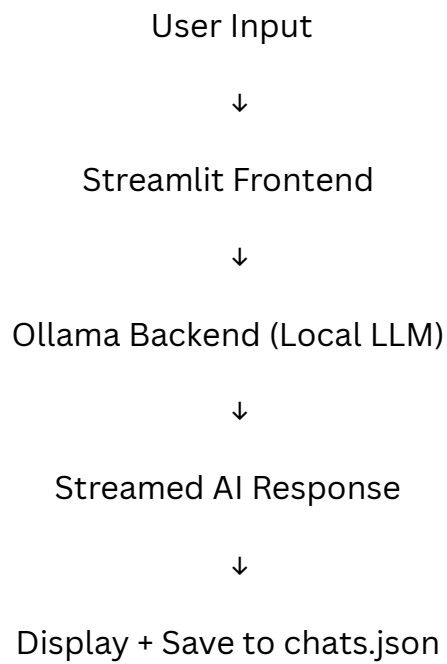
- Integrate a local AI model for code generation and explanation.
- Provide a user-friendly web interface using Streamlit.
- Maintain multi-session persistent chat history.
- Handle model loading, error detection, and RAM management gracefully.

5.Technologies Used

Component	Technology / Library	Purpose
Frontend	Streamlit (Python)	Build interactive and real-time chat UI
Backend AI Engine	Ollama (Local LLMs: llama2:7b, llama2:latest)	Handle code generation and explanation tasks
Data Storage	JSON file (chats.json)	Store persistent chat history
UUID Library	uuid	Generate unique identifiers for each chat session
Datetime	datetime	Capture timestamps for chat creation and messages
JSON Module	json	Read/write chat data in structured format

OS Module	os	Interact with local file system for persistence
Subprocess	subprocess	Execute system commands like ollama list
Requests	requests	Communicate with Ollama API to check connectivity
Psutil	psutil	Monitor system RAM and manage memory warnings

6.System Architecture



Components:

- **Frontend:** Streamlit
- **Backend:** Ollama (local models: llama2, gemma, mistral)
- **Data Storage:** JSON

7.Ollama Integration

Ollama is the **core backend AI engine** in the CodeGen AI & Code Explainer project.□

It is responsible for **handling all natural language and code generation tasks** through locally hosted **Large Language Models (LLMs)** such as llama2:7b, mistral, and codellama.

Unlike cloud-based APIs (like OpenAI), Ollama runs **completely offline** on your system.□

This ensures **privacy, zero network dependency, and faster local inference.**

Purpose of Ollama Integration

- To **generate and explain code** using local AI models.
- To **stream responses in real-time** for a conversational feel.
- To **list and select available models** dynamically.
- To **check server connectivity** before querying the model.
- To **gracefully handle model or memory errors.**

Integration Steps

Step 1: Check if Ollama Server is Running

```
def is_ollama_running():  
    try:  
        r = requests.get("http://localhost:11434/api/tags", timeout=2)  
        return r.status_code == 200  
    except:  
        return False
```

This function sends a **GET request** to Ollama's API endpoint (/api/tags).

If it gets a **200 (OK)** response, Ollama is running. If it fails, it means Ollama isn't started yet, and a warning is displayed in stremlit

```
def is_ollama_running():  
    try:  
        result = subprocess.check_output(["ollama", "list"], text=True)  
        return True  
    except Exception:  
        warning("⚠️ ollama is not running. Start it with:\n```powershell\nollama run llama2\n```")  
        return False
```

Step 2: Retrieve Installed Models Dynamically

To allow users to select from models already downloaded, the app uses the subprocess module to run ollama list.

```
def get_installed_models():  
    try:  
        result = subprocess.check_output(["ollama", "list"], text=True)  
        models = []  
        for line in result.splitlines()[1:]:  
            parts = line.split()  
            if parts:  
                models.append(parts[0])  
        return models or ["llama2"]  
    except Exception:  
        return ["llama2"]
```

- Runs the command ollama list in the background.
- Parses the output to extract model names (like llama2:7b, mistral).
- Returns them as a **list** for the user to select from in the sidebar.
- Ensures fallback to "llama2" if any error occurs.

In the **Streamlit sidebar**, these models are displayed dynamically:

```
available_models = get_installed_models()  
model = st.selectbox("Choose model:", available_models, index=0)
```

Step 3: Querying Ollama (Generating or Explaining Code)

When the user enters a message in the chat box, it is sent to Ollama's local API for inference.

```
stream = ollama.chat(model=model, messages=chat["messages"], stream=True)
for chunk in stream:
    content = chunk["message"]["content"]
    if content:
        assistant_response += content
        placeholder.markdown(assistant_response + "|")
```

ollama.chat() sends all previous chat messages to the chosen model.

stream=True enables **real-time response streaming**.

Each chunk represents a small part of the model's output.

The content is appended to assistant_response, which updates dynamically in the UI using:

Once the full response is complete :

```
placeholder.markdown(assistant_response)
chat["messages"].append({"role": "assistant", "content": assistant_response})
save_chats(st.session_state["chats"])
```

This saves the assistant's final response to chat history (chats.json).

Step 4: Handling Model Errors and Memory Warnings

When the selected model (like llama2:7b) requires more system RAM than available, Ollama throws a status code: 500 error.

```
st.warning(
    f"⚠️ Model **{model_name}** requires ~{required} GB RAM, "
    f"but your system only has {system_ram} GB. Switching to a lighter model..."
)
```

To handle this gracefully, a **RAM check function** is implemented:

```

# RAM Check
# -----
def get_system_ram_gb():
    return round(psutil.virtual_memory().total / (1024 ** 3), 1)

def check_model_ram(model_name):
    ram_requirements = {
        "llama2:7b": 6.0,
        "llama2:13b": 12.0,
        "llama2:70b": 64.0,
        "mistral": 4.0,
        "gemma:2b": 2.0,
    }
    required = ram_requirements.get(model_name, 2.0)
    system_ram = get_system_ram_gb()

    if system_ram < required:
        st.warning(
            f"⚠ Model **{model_name}** requires ~{required} GB RAM, "
            f"but your system only has {system_ram} GB. Switching to a lighter model..."
        )
        return False
    return True

```

Automatically detects available RAM.

If insufficient memory, displays a **warning** and switches to a **lighter model** like gemma:2b or mistral.

Step 5: Saving Chat Data Locally

All conversation data is stored in a file named chats.json for persistence.

```

def save_chats(chats):
    with open(DATA_FILE, "w", encoding="utf-8") as f:
        json.dump(chats, f, indent=2)

```

This allows users to:

- Resume old chats.
- View previously generated code.
- Keep a learning log of AI interactions.

Why Ollama Was Chosen

Feature	Reason
Local Deployment	Works without an internet connection.
Data Privacy	No data leaves the system.
Multiple Model Support	Compatible with Llama2, Mistral, Gemma, CodeLlama.
Streaming API	Allows real-time typing effect like ChatGPT.
Simple Integration	Easy Python bindings using ollama package.

8 .Streamlit Frontend Design

Sidebar:

- Chat history (searchable)
- Model selector
- New chat button

Main Window:

- Displays user and assistant messages
- Real-time AI streaming

```
if prompt := st.chat_input("Type your message..."):
    if not prompt.strip():
        st.warning("Please enter a message before sending.")
        st.stop()
```

provides dynamic, reactive UI using `st.session_state`.

9. Data Management (JSON Storage)

```
data.json > ...
[
  {
    "id": "bfdfd729439d46dc832b86c9345b8275",
    "title": "New Chat",
    "messages": [],
    "created_at": "2025-10-03T20:05:57.347152"
  }
]
```

Enables multi-session persistence.□

Supports search, delete, and title update.

10. Features

Feature	Description
Multi-chat sessions	Handle multiple conversations
Model selection	Switch between Ollama models
Streaming AI responses	Real-time output
Persistent history	Chats stored in chats.json
Auto RAM detection	Switches to lighter model
Error alerts	Detects if Ollama not running
Search & filter	Case-insensitive chat lookup

11. Challenges & Error Fixes

Issue Faced	Error Message / Cause	Solution Implemented
1. Ollama model failed to load	Error: model requires more system memory (6.0 GiB) than available (5.4 GiB)	Added RAM check function using psutil to auto-switch to lighter models like gemma:2b or mistral.
2. Ollama not running	❑ Error talking to Ollama	Implemented <code>is_ollama_running()</code> check before sending chat request. Displays user-friendly warning with startup command.
3. Streamlit permission issue	[Errno 13] Permission denied: '..venv\\Scripts\\python.exe'	Recreated virtual environment outside restricted directories (moved from OneDrive to local folder).

4. Model not found	Error: pull model manifest: file does not exist	Used command ollama list to confirm available models and pulled specific versions manually.
5. Repeated model crash	Llama2 7B too heavy for 8GB RAM	Optimized by adding auto RAM detection → switches to mistral automatically.
6. Chat history not saving	JSONDecodeError on load	Wrapped file reading in try-except and reinitialized empty list if JSON corrupt.
7. Stream not updating in real time	Streamlit not refreshing output dynamically	Used st.empty() placeholder to simulate typing indicator during streaming.

These solutions ensured **smooth real-time AI interaction**, stable local operation, and a **seamless frontend-backend workflow**

12. Deployment Steps

Step 1 – Install Required Packages

`pip install streamlit ollama requests psutil`

Step 2 – Start Ollama Service

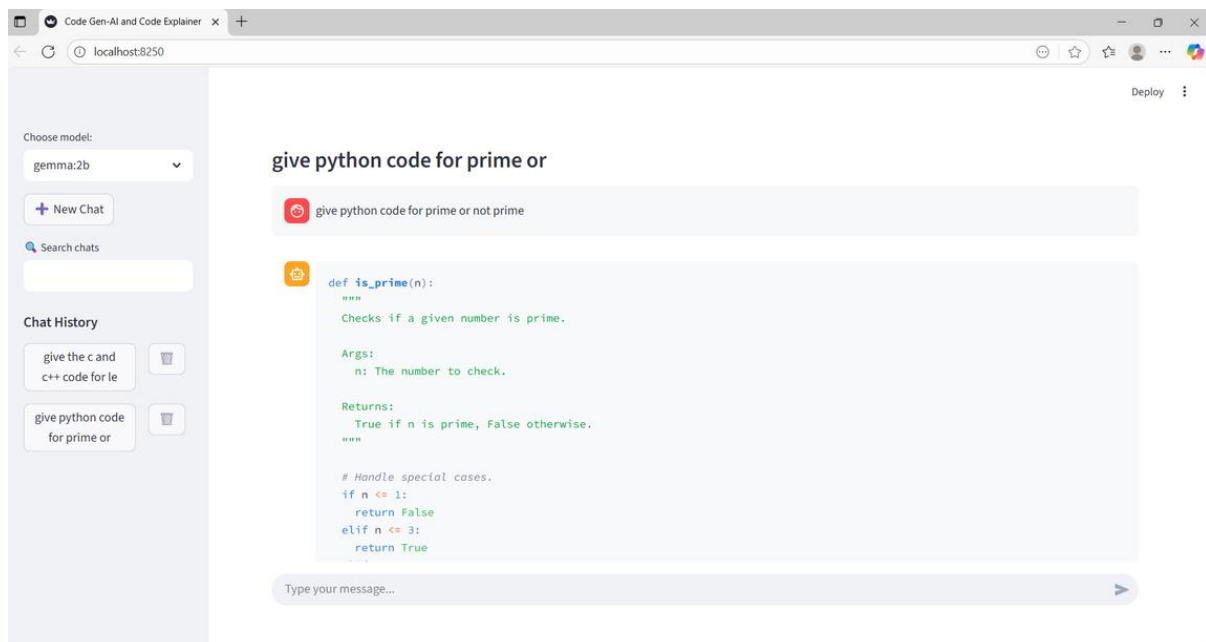
`ollama run llama2`

Step 3 – Run Streamlit App

`streamlit run app.py --server.port 8502`

Step 4 – Access Application

Open browser → <http://localhost:8502>



13. Key Learnings

- Learned **LLM integration with Streamlit**.
 - Understood how **local AI models** (Ollama) operate without cloud.
 - Improved understanding of **real-time streaming & UI state management**.
 - Solved practical issues with **RAM optimization and permission handling**.
- Enhanced debugging and **error resilience** in AI app pipelines.

14. Conclusion

The project successfully integrates:

- **Streamlit UI** (for interaction)
- **Ollama Backend** (for local AI processing)
- **JSON** (for data persistence)

The **CodeGen AI & Code Explainer** provides:

- Real-time AI-driven responses
- Multi-session support
- Offline operation
- Developer-friendly design

It demonstrates how **local LLMs** can power smart, private, and high-performance AI assistants.

15. References

[Streamlit Documentation](#)

[Ollama Documentation](#)

Python Libraries: uuid, datetime, json, os, subprocess, requests, psutil

